Distributionally Robust Group Relative Policy Optimisation

Finlay Sanders [1] Sam Turner [1] Debmalya Mandal [1,2]

[1]Department of Computer Science, University of Warwick, United Kingdom [2]Project Supervisor

Introduction

In recent years, **large language models** (LLMs) have massively grown in capability across a diverse set of tasks. Often a single language model is used in a variety of problem domains, user contexts and prompting styles with the assumption that performance standards will hold. In practice, LLMs can struggle to generalise beyond the scope of their training data, which has major implications for performance in real-world tasks that differ from the training set.

Group Relative Policy Optimisation (GRPO) in particular has proven effective in enhancing mathematical reasoning and code generation of LLMs [1] [2]. By estimating baselines from a group of completions for each prompt, GRPO removes the need for a critic model and vastly reduces the required training resources.

The focus of our project is mitigating the effects of the disparity between prompts seen in training and prompts seen in testing, which is known as "**distribution shift**". We aim to fine-tune models with GRPO such that they maintain their performance under a prompt distribution that differs from the one seen in training. By extending the work of Mandal et al. [3], which introduces distributionally robust versions of reward-based RLHF and reward-free DPO, with a distributionally robust version of GRPO, we aim to improve generalisation across tasks like mathematical reasoning and code generation in several programming languages.

Reinforcement Learning

Reinforcement Learning (RL) focuses on teaching agents a task through trial and error. RL has more traditionally been used in cases such as robotics, which can be easily simulated. In recent years, RL has become an important part of fine-tuning large language models (LLMs) using methods such as **Reinforcement Learning from Human Feedback** (RLHF) or GRPO.

RL is characterised based on interactions between the **agent** and the **environment** [4]. In traditional cases, the agent could be a robot but when fine-tuning language models, the "agent" is the model and the "environment" usually becomes the prompt distribution or a reward model. The agent makes an **action** a based on its (possibly partial) **observation** o of the environment **state** s. The agent receives a **reward** signal, allowing it to associate these actions with reward, and a **policy** π to choose which action it can take, which is usually denoted as $a_t \sim \pi_\theta(\cdot|s_t)$. These policies are based on a set of **parameters** θ (weights and biases of a neural network) which we can tune to change the agent's behaviour. The reward at a step $r_t = R(s_t, a_t)$ is defined based on the current state, which we use to find the **discounted return** of a **trajectory** $\tau = (s_0, a_0, s_1, a_1, \ldots)$ with:

$$R(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t$$

where $\gamma \in (0,1)$ is the **discount factor** which reduces the influence of reward obtained far in the future. As such, the goal of RL is to maximise the expected return in order to find the **optimal policy** π^*. To optimise a policy, we use an algorithm such as **Proximal Policy Optimisation** (PPO) which performs many steps of **Stochastic Gradient Descent** (SGD) to maximise the **objective**:

$$\ell_{\text{PPO}} = -\min\left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A^{\pi_{\theta_{\text{old}}}}(s,a),\ \text{clip}\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)},\ 1-\epsilon,\ 1+\epsilon\right) A^{\pi_{\theta_{\text{old}}}}(s,a)\right]$$

where ϵ is a **hyperparameter** which restricts the new policy diverging too far from the previous one. PPO requires the **advantage** A of an action, which describes how much *better* it is than others on average. This is computed using a **value function** which is learnt alongside the policy. When training LLMs, learning a value function is very expensive. We instead use **GRPO** by estimating baselines from a group G of generated **completions** for the same **prompt**:

$$\ell_{\text{GRPO}}(\theta, x) = -\frac{1}{G}\sum_{i=1}^{G} \min\left[r_i(\theta)\, A_i,\ \text{clip}\left(r_i(\theta),\ 1-\epsilon,\ 1+\epsilon\right) A_i\right] + \beta\, D_{\text{KL}}\big(\pi_\theta(\cdot|x) \,\|\, \pi_{\text{ref}}(\cdot|x)\big)$$

Distributionally Robust Optimisation

In many decision problems we face uncertainty about the probability distribution of future outcomes. **Distributionally Robust Optimisation** (DRO) addresses this by choosing decisions that perform well under the *worst-case* distribution which is consistent with our dataset.

We formalise distributional uncertainty via an **ambiguity set** around the empirical distribution $\hat{P}$, measured using **total variation** (TV) distance.

$$\text{TV}(P, Q) = \sup_{A \in \mathcal{F}} \big|P(A) - Q(A)\big|$$

Intuitively, if $\text{TV}(P, \hat{P}) = \rho$, then the probability of any event can differ by at most ρ between P and $\hat{P}$.

Our ambiguity set *around the training distribution* is then:

$$\mathcal{U}_\rho(\hat{P}) = \left\{ P : \text{TV}(P, \hat{P}) \le \rho \right\}$$

Given a loss ℓ, the distributionally robust optimization problem becomes:

$$\min_\theta \max_{P \in \mathcal{U}_\rho(\hat{P})} \mathop{\mathbb{E}}_{x \sim P}\big[\,\ell(\theta, x)\,\big]$$

This optimises for parameters θ which **minimise the worst-case expected loss** over all possible distributions which are within TV-distance ρ of the training distribution. This reflects the fact that test-time inputs may differ from the training data.

Distributionally Robust GRPO (Ours)

Mandal et al. [3] introduce distributionally robust versions of reward modelled Reinforcement Learning from Human Feedback (RLHF), as well as Direct Preference Optimisation (DPO) which learns without a reward model using preferences. **We extend this technique to Group Relative Policy Optimisation** (GRPO) by applying the same principles.

Specifically, we formulate the distributionally robust problem as:

$$\min_\theta \sup_{D:\, d_\phi(D, D_{\text{src}}) \le \rho} \mathop{\mathbb{E}}_{x \sim D}\big[\ell_{\text{GRPO}}(\theta, x)\big]$$

We propose a minibatch gradient descent based algorithm for solving this problem:

Algorithm: Minibatch Distributionally Robust GRPO (DR-GRPO)

Require: Source dataset $\mathcal{D}_{\text{src}}$, batch size n, radius ρ, step size η, initial policy parameters θ

repeat

Sample a minibatch of prompts $B = \{x_i\}_{i=1}^n$ from $\mathcal{D}_{\text{src}}$

Compute per-example GRPO losses $\ell_i \leftarrow \ell_{\text{GRPO}}(\theta, x_i)$ for $i = 1, \ldots, n$

Compute adversarial weights

$$q^\star \leftarrow \arg\max_{q \in \Delta_n :\, TV\left(q, \frac{1}{n}\mathbf{1}\right) \le \rho} \sum_{i=1}^{n} q_i\, \ell_i$$

Form the robust minibatch loss

$$\widehat{\mathcal{L}}_{\text{DR-GRPO}}(\theta) \leftarrow \sum_{i=1}^{n} q_i^\star\, \ell_{\text{GRPO}}(\theta, x_i)$$

Update policy parameters with a gradient step

$$\theta \leftarrow \theta - \eta\, \nabla_\theta\, \widehat{\mathcal{L}}_{\text{DR-GRPO}}(\theta)$$

until convergence

The algorithm is designed to be easily integrated into existing fine-tuning pipelines, requiring only a reweighing of the minibatch losses.

Experiments

In our experiments, we fine-tune the `Qwen/Qwen2.5-0.5B-Instruct` [5] (494M parameters) and `google/gemma-2-2b-it` [6] (2B parameters) models on the `AI-MO/NuminaMath-TIR` [7], `openai/gsm8k` [8] and `google-research-datasets/mbpp` [9] datasets using Distributionally Robust GRPO. We also use **LoRA** (Low-Rank Adapters) with rank $r = 8$, $\alpha = 32$, and dropout 0.1 on the query and value projection layers, which results in fine-tuning **0.11% of parameters** for computational efficiency. This allows us to perform the fine-tuning process on a single 40GB **NVIDIA A100 SXM4** GPU.

Our goal is to **test the out-of-distribution performance** of this model compared to a baseline which is fine-tuned on the same dataset with vanilla GRPO. For the reward function, we utilise a binary reward function $R(y, y^*) = R_{\text{format}}(y) \times R_{\text{accuracy}}(y, y^*)$ in `trl` [10] where:

$$R_{\text{format}}(y) = \begin{cases} 1 & \text{if pattern matches} \\ & \texttt{<think>...</think>} \\ & \texttt{<answer>...</answer>} \\ 0 & \text{otherwise} \end{cases} \qquad R_{\text{accuracy}}(y, y^*) = \begin{cases} 1 & \text{if verify}(\text{extract}(y), y^*) \\ 0 & \text{otherwise} \end{cases}$$

Here, y is the model completion and y^* is the ground truth solution. Verification uses symbolic math parsing via the `math_verify` library.

Parameter	Value	Parameter	Value
Learning rate	10^{-5}	Max completion length	64 tokens
Batch size	128 completions	Max prompt length	128 tokens
Prompts per batch	16	Precision	bf16
Generations per prompt	8	Clipping parameter ϵ	0.2
KL penalty β	0.01	LoRA rank r	8
LoRA α	32	LoRA dropout	0.1

Table 1. Training configuration

Results and Reflections

We ran a number of experiments fine-tuning on maths problems and evaluating on programming problems and vice-versa, with the **goal of improved generalisation across tasks**. Unfortunately, we did not see out-of-distribution benefits within the time frame of the project. We suspect this is because of the nature of our binary reward function. This is because in distributionally robust optimisation, we **trade off performance in the empirical data distribution for better generalisation** across distributions by optimising for the "worst case". However, with a a binary reward function, the "worst case" is the same as the "average case", so DRO provides limited benefits. We also found that when these **binary rewards were sparse**, DRO would "squash" the already small training signal by smearing weights from successful completions to failed completions. With more time, we would explore ways to modify the algorithm to support this binary training format which is common in GRPO fine tuning.

References

[1] Zhihong Shao, Peiyi Wang, Qihao Zhu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

[2] DeepSeek-AI, Daya Guo, Dejian Yang, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948.

[3] Debmalya Mandal, Paulius Sasnauskas, and Goran Radanovic. Distributionally robust reinforcement learning with human feedback, 2025. URL https://arxiv.org/abs/2503.00539.

[4] Joshua Achiam and OpenAI. Spinning up in deep reinforcement learning. https://spinningup.openai.com/, 2018.

[5] An Yang, Baosong Yang, Binyuan Hui, et al. Qwen2 technical report, 2024. URL https://arxiv.org/abs/2407.10671.

[6] Gemma Team, Morgane Riviere, Shreya Pathak, et al. Gemma 2: Improving open language models at a practical size, 2024. URL https://arxiv.org/abs/2408.00118.

[7] Jia Li, Edward Beeching, Lewis Tunstall, et al. Numinamath. [https://github.com/project-numina/aimo-progress-prize](https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf), 2024.

[8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, et al. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

[9] Jacob Austin, Augustus Odena, Maxwell Nye, et al. Program synthesis with large language models, 2021. URL https://arxiv.org/abs/2108.07732.

[10] Quentin Gallouédec. Grpo trainer, 2025. URL https://huggingface.co/docs/trl/main/en/grpo_trainer. TRL documentation (branch: main).